

Viking: A Modern Platform for Ultrastructural Connectome Mapping and Analysis

Methods paper draft. This document expands the original framework described by Anderson et al. (2009), “A Computational Framework for Ultrastructural Mapping of Neural Circuitry”, PLoS Biology 7(3):e1000074, to reflect the substantial development of the Viking platform in the years since publication. It is intended as a starting draft for a peer-reviewed methods manuscript and for a colleague’s methods paper on 3D connectome analysis. Citation markers in the form [CITE: ...] mark places where the authors should insert formal references; technical claims are grounded in the current source tree and noted with file paths.

Abstract

The reconstruction of neural circuits from serial-section electron microscopy requires software that can assemble terabyte-scale image volumes, support the collaborative annotation of cells and their connections, and convert those annotations into quantitative anatomical and network models. The original Viking framework (Anderson et al., 2009) established an end-to-end pipeline for acquiring, registering, and viewing such volumes, together with a client for slice-by-slice annotation. Since then the platform has been substantially re-engineered. The desktop client has been rebuilt on a modern real-time graphics stack; annotation geometry is now stored as indexed spatial types in a relational database; programmatic access is provided through an OData service, with a gRPC service under development; user access is mediated by a dedicated identity service with volume-scoped permissions; and a reconstruction pipeline converts two-dimensional annotations into three-dimensional surface meshes and connectivity graphs suitable for downstream analysis. This paper documents the current architecture, the data model, and the analysis pipeline, with emphasis on the components that did not exist when the framework was first described.

1. Introduction

The 2009 framework paper addressed a then-novel problem: how to turn the raw output of automated serial-section transmission electron microscopy into a navigable, annotatable volume at a scale of terabytes [CITE: Anderson 2009]. Its contributions spanned acquisition with SerialEM [CITE: Mastronarde], image mosaicking and slice-to-slice registration with the *ir-tools* suite, color and intensity correction, volume assembly, and a Win32 client (“Viking”) that paged through reconstructed sections over HTTP and let annotators mark structures and their connections.

That framework remains sound, and the acquisition and registration stages are largely unchanged. What has changed is everything downstream of a registered volume. Connectome projects now involve more annotators working concurrently, larger and more numerous volumes, and a strong demand for quantitative outputs: not just “which cells connect to which”, but the three-dimensional morphology of each process and the spatial distribution of its synapses. Meeting these demands required changes that the original paper did not anticipate: a spatially indexed database so the client can load only the annotations within the current viewport; service APIs so that analysis code can read the annotation graph without screen scraping; an authentication layer so that volumes can be shared across institutions with appropriate permissions; and an automated pipeline that lifts flat, per-section annotations into closed 3D surfaces.

This paper documents the platform as it exists today. Where a stage is unchanged from 2009 we summarize it briefly and refer to the original work; where a stage is new or substantially revised we describe it in enough detail to support methods reporting and reproduction.

2. Image volume pipeline

2.1 Acquisition, mosaicking, and registration

Image acquisition, tile mosaicking, intensity correction, and slice-to-slice registration follow the pipeline described in 2009 and are only summarized here. Sections are imaged as overlapping tiles; tiles are assembled into a per-section mosaic; and consecutive mosaics are registered to one another to build a coherent volume. The registration between any two sections is captured as a “slice-to-slice” (STOS) transform.

2.2 Volume model

In the current client, a dataset is described by a VikingXML document that the `Volume` class parses (`Clients/VolumeModel/Volume.cs`). A `Volume` holds an ordered set of `Section` objects (`Clients/VolumeModel/Section.cs`), the groups of STOS transforms that register them, the host URL, and local cache paths. Each section exposes one or more image pyramids (`Pyramid`, `Clients/VolumeModel/Pyramid.cs`), which map a downsampling level to the directory of tiles at that resolution.

Two tile-mapping strategies coexist. `FixedTileCountMapping` (`Clients/VolumeModel/FixedTileCountMapping.cs`) models classic EM mosaics, where each pyramid level has a fixed number of tiles of varying size, driven by per-tile `.mosaic` transforms. `TileGridMapping` models pre-tiled pyramids with a fixed pixel size per tile and a grid that grows with resolution, which is also used for tile servers in the style of the Open Connectome Project. At render time, the visible tiles for the current view frustum are gathered into a `TilePyramid` (`Clients/VolumeModel/TilePyramid.cs`), and each becomes a `TileViewModel` carrying mesh vertices, texture coordinates, and a texture URL.

2.3 Tile serving and caching

Tiles and metadata are fetched over HTTP through a shared client. Tile images are loaded by `TextureReaderV2` (`Clients/Viking/NGVV/Threading/TextureReaderV2.cs`), which checks a local disk cache before requesting from the server and writes fetched tiles back to disk. Decoded tiles are held in an in-memory, time-ordered cache (`TileCache`, `Clients/VolumeModel/TileCache.cs`) that is trimmed periodically to bound memory use. This multi-layer caching is what allows fluid navigation through volumes far larger than memory.

3. Coordinate spaces and registration

3.1 Two coordinate spaces

A central concept, only implicit in the 2009 paper, is that Viking maintains two distinct two-dimensional coordinate spaces and converts between them constantly:

- **Mosaic (section) space** is the native coordinate system of a single assembled section, after tile mosaicking but before slice-to-slice registration. Annotations are authored and stored in this space.
- **Volume space** is the common, registered coordinate frame shared across all sections. Cross-section navigation, adjacent-section overlays, and 3D morphology all live in this space.

The distinction is documented for developers in `.cursor/rules/coordinate-spaces.mdc`, and the database stores each annotation footprint in both spaces (Section 5).

3.2 Transform types

STOS transforms are represented by a family of classes in `Geometry/Transforms/`: `RigidTransform` for rotation and translation, `GridTransform` and `MeshTransform` for discrete warps, `RBFTransform` for a continuous radial-basis fallback, and `TriangulationTransform` as the base for triangulated warps. Transforms are read from `.stos` files (from disk, HTTP, or zip archives) by `TransformFactory`, with provenance recorded in `StosTransformInfo` (control section, mapped section, and modification time). By convention the control section corresponds to the volume side and the mapped section to the mosaic side.

3.3 From pairwise to global registration

VikingXML lists the pairwise STOS files. The `RegistrationTree` (`Clients/VolumeModel/RegistrationTree.cs`) arranges these pairwise relationships into a tree, and `Volume.CreateVolumeTransforms` walks the tree to compose, for each section, the chain of transforms that maps it into the volume reference frame. Composed transforms are cached to disk to avoid recomputation. Tile rendering adds a second warp stage: `SectionToVolumeMapping` composes each per-tile mosaic transform with the section's STOS transform, so individual tiles are triangulated through the registration rather than merely repositioned.

3.4 Runtime conversion and the transform cache

At runtime, the interface `IVolumeToSectionTransform` exposes `SectionToVolume` and `VolumeToSection` (with bounds-safe Try variants). The client uses these to convert the visible volume rectangle into mosaic-space bounds before querying the database, and to place annotations correctly on screen.

Composed transforms are serialized to a JSON cache by `JsonTransformSerializer` (`Clients/VolumeModel/JsonTransformSerializer.cs`), which replaced an older binary format. The serializer uses a polymorphic converter to handle the transform class hierarchy. A 2026 revision separated the read and write option sets so that deserialization does not re-enter the computed-property converter, eliminating a stack overflow that occurred when large cached section mappings were loaded on section change.

4. The annotation workflow

The 2009 paper described annotation as a roughly twelve-step manual process of marking structures section by section. That basic loop remains, but the mechanics have changed in three important ways: annotations are loaded by spatial region rather than by whole section; structures and locations are first-class linked entities; and adjacent sections are displayed simultaneously to support tracing continuity.

An annotator navigates to a section, and the client loads the structures and locations whose footprints intersect the current viewport. New annotations are drawn with a markup tool appropriate to the structure type (a point, a line, or a polygon). As a process is traced from section to section, successive footprints are linked to express physical continuity, and connections between distinct structures (for example, a synapse between two cells) are expressed as structure links. The following sections describe the data model and client that make this workflow efficient at scale.

5. Annotation data model

5.1 Core entities

The annotation database is built around four entities, defined in `Servers/ConnectomeDataModelCore/` and mirrored on the client in `Clients/WebAnnotationModel/`:

- **Structure** – a logical biological object: a cell, a process, an organelle, or a synapse. A structure has a `StructureType` and may have a parent structure, which is how, for example, a synapse is modeled as a child of the cell that contains it.
- **Location** – the two-dimensional footprint of a structure on a single section. Locations carry shape information and flags such as terminal (the end of a process) and off-edge (the process leaves the imaged volume).
- **StructureLink** – a connectivity edge between two structures, directional or bidirectional; this is the substrate of the connectome graph.
- **LocationLink** – a continuity link between two locations, typically on adjacent sections, expressing that they are the same process traced across the cut.

5.2 Types, markup, and permitted links

Each structure has a `StructureType` (`Clients/WebAnnotationModel/Objects/StructureTypeObj.cs`) that defines its name, code, display color, hotkey, and default markup geometry (point, line, or polygon). Structure types form their own parent/child hierarchy. To keep annotation biologically meaningful, the schema records *permitted* structure links: which source type may connect

to which target type, and in which direction ([Clients/WebAnnotationModel/Objects/PermittedStructureLinkObj.cs](#)). The client uses these constraints to validate connections as they are drawn.

5.3 Spatial geometry

The most consequential schema change since 2009 is the migration of annotation geometry to native SQL Server spatial types. Before August 2015, a footprint was stored as scalar `x`, `y`, and `Radius` columns plus a vertex list for polygons. On 19 August 2015 (database schema versions 24 and 25, commit `ffe9cceb`), the `Location` table gained two geometry columns:

- `MosaicShape` – the footprint in mosaic (section) space.
- `VolumeShape` – the footprint in registered volume space.

Circular footprints were converted to `CURVEPOLYGON(CIRCULARSTRING(...))` and point footprints to `POINT(...)`. The scalar centroid columns were retained as persisted computed columns derived from the geometry, for example:

```
ALTER TABLE Location ADD X as ISNULL(MosaicShape.STCentroid().STX, ISNULL(  
◀ | | ▶
```

The full migration is in [Servers/SQL/DatabaseCreateUpdate/CreateUpdateDatabase.sql](#) (version 24 and 25 blocks, approximately lines 2368-2440), and is described for citation in [spatial-columns-factsheet.md](#). A spatial index was added a week later (26 August 2015), and queries were updated to operate in either mosaic or volume coordinates in March 2016. Storing geometry natively is what makes region-based annotation loading (Section 6.2) efficient, because the database can answer “which footprints intersect this rectangle on this section” using a spatial index rather than a full scan.

5.4 Schema versioning and change tracking

The database is created and upgraded by a single idempotent script that applies numbered migrations and records each in a `DBVersion` table; the script currently reaches version 83. Change Data Capture is enabled on annotation tables, which supports auditing and downstream replication. Region queries are served by stored procedures such as `SelectSectionLocationsAndLinksInMosaicBounds`, which returns the locations and links within a bounding box on a section as of a given query time, enabling consistent reads while other annotators are editing.

6. Viking client architecture

6.1 Rendering and navigation

The client has been rebuilt on the MonoGame/XNA real-time graphics stack, with a shared graphics library ([Clients/MonogameXNAGraphicsShared/](#)) reused across tools. The section viewer ([Clients/Viking/NGVV/UI/Controls/SectionViewerControl.cs](#)) pages through sections in volume order, preloading the immediately adjacent sections, and supports keyboard, mouse, and pen input for stepping and zooming. Users designate reference sections above and below the current one, which drives the adjacent-section overlays used during tracing.

6.2 Region-based annotation loading

Rather than loading all annotations on a section, the client loads only those within the current view. The `RegionLoader` ([Clients/WebAnnotationModel/RegionLoader/RegionLoader.cs](#)) divides space into a pyramid of grid cells and issues asynchronous queries per cell as the view changes, refreshing on an interval so that other annotators’ edits appear. The visible volume rectangle is converted to mosaic-space bounds through `IVolumeToSectionTransform`, the bounds query is sent to the server, and results are held locally in an R-tree for fast intersection and radius tests. The overlay loads not only the current section but a configurable window of nearby sections, prioritized by distance from the current depth.

6.3 Rendering annotations

Annotations are drawn by an overlay ([Clients/Viking/WebAnnotation/UI/AnnotationOverlay.cs](#)) that composes several view layers: filled and outlined footprints on the current section; footprints from adjacent sections shown with distinct styling so an annotator can follow a process across the cut; location links that connect a footprint to its continuation on the neighboring section; structure links drawn between connected structures; and text labels. A depth/stencil pass ensures that annotation fills do not obscure the underlying electron micrograph.

7. Server infrastructure and APIs

7.1 Annotation service (WCF and gRPC)

Annotations are read and written through a service layer. The established implementation is a WCF service ([Servers/AnnotationService/](#)) exposing contracts for structures, structure types, locations, permitted links, circuit queries, and volume metadata, with role-based authorization (Read, Write, Annotate, Review, Modify). A gRPC replacement is under active development ([Servers/GrpcAnnotationService/](#)), with protocol definitions for the same domain under [gRPCAnnotationServiceTypes/Protos/](#). The migration to gRPC is ongoing; both services target the same database.

7.2 OData read API

For programmatic analysis, the platform exposes an OData v4 read API ([Servers/ConnectomeODataV4/](#)). It publishes entity sets for structure types, structures, locations, structure links, permitted structure links, location links, and spatial caches, and it offers bound functions for connectivity analysis, including `Network` (the subgraph reachable from a set of structures within a number of hops), `NetworkLinks`, and `Scale` (the physical scale of the volume). This API is the primary entry point for the morphology and network pipeline described in Section 9, and it is well suited to external analysis code, including a collaborator's 3D connectome analysis.

7.3 Export service

A dedicated export service ([Servers/DataExport/](#)) builds graphs through the same factories used internally and serializes them to standard formats: DOT, TLP, GraphML (GML), and JSON for network and motif graphs, and TLP and JSON for morphology graphs. These formats let connectome graphs be loaded directly into common graph-analysis and visualization tools.

7.4 Authentication and authorization

Access is mediated by a dedicated identity service based on the IdentityServer framework, comprising a token issuer, a query API for users and rights, and a management site for accounts and volume permissions. Authorization is volume-scoped: claims take the form `{VolumeName}. {permission}` (for example, `RC1.Read`), and the annotation service validates the bearer token's issuer and audience and checks the required claim before serving a request. This is what allows a single deployment to host multiple volumes for multiple institutions with appropriate separation.

7.5 Deployment

Servers are deployed with Docker. A combined Windows/IIS image packages the WCF annotation service, the OData API, and the export service behind a single container, exposing them under `/annotation`, `/odata`, and `/dataexport` respectively; the gRPC and segmentation services run as Linux containers. Runtime configuration and secrets are mounted from host directories following the convention `D:\Docker\mounted-configs\<ServiceName>` and `D:\Docker\Builds\<ServiceName>`, documented in [_cursor/rules/docker-config-and-builds-location.mdc](#).

8. Segmentation and assisted tracing

Beyond manual tracing, the platform integrates a segmentation service over gRPC that wraps a Segment Anything (SAM2) model ([gRPC_Protos/Segmentation/SAM2/segmentation.proto](#)), hosted as a separate

container. The intent is to accelerate the most laborious part of annotation, the drawing of accurate polygon boundaries, by proposing segmentations that an annotator can accept or correct. This service is separate from the annotation CRUD path and is deployed independently.

9. Three-dimensional morphology and connectome analysis

9.1 From annotations to graphs

The analysis pipeline begins by reading annotations into in-memory graphs through [AnnotationVizLib/](#). Two graph abstractions are central. A `MorphologyGraph` represents one or more structures at the level of geometry: its nodes are locations (each a spatial geometry plus a Z depth) and its edges are location links. A `NeuronGraph` represents connectivity at the level of structures, where nodes are cells and edges are structure links. The same data can be fetched through three interchangeable backends – an OData client (`ODataMorphologyFactory`), the WCF service (`WCFMorphologyFactory`), and a lightweight OData client (`SimpleODataMorphologyFactory`) – so analysis code is not tied to a single transport.

9.2 Surface reconstruction

To produce three-dimensional surfaces, the per-section footprints of a structure must be tiled into a closed mesh. This is handled in [MorphologyMesh/](#). A `SliceGraph` groups the connected locations of a morphology into slices between adjacent Z levels. The `BajajMeshGenerator` ([MorphologyMesh/Generators/BajajMeshGenerator.cs](#)) then reconstructs a surface between consecutive contours using a branching-aware tiling approach: it computes a constrained Delaunay triangulation of the paired contours, builds a region graph, selects optimal tiling vertices, and generates validated “slice chords” that connect contour pairs while respecting branching and avoiding self-intersection [CITE: Bajaj contour tiling]. The resulting mesh can be exported to COLLADA via `MorphologyColladaView` for visualization and for import into 3D analysis tools. The interactive testbeds `BajajTest` and `BajajMultiTest` ([Clients/MonogameTestbed/BajajTest.cs](#)) visualize the reconstruction stage by stage and were used to validate the algorithm.

9.3 Network export for analysis

In parallel with morphology, the connectivity graph is exported for quantitative network analysis through the export service (Section 7.3), in DOT, TLP, GraphML, and JSON. This is the natural hand-off point to external connectome analysis methods, including studies of network motifs and 3D spatial organization.

10. Quality control, performance, and scale

Several mechanisms support data quality and performance at scale. `VikingAU` ([Clients/VikingAU/](#)) automatically adjusts annotation positions in the database, correcting systematic offsets. Spatial indexes and bounds stored procedures keep region queries fast even as the number of annotations grows into the millions. The tile and transform caches (Sections 2.3 and 3.4) bound client memory and network use. Query-date semantics in the region stored procedures give annotators a consistent view of the data while edits proceed concurrently, and Change Data Capture provides an audit trail.

11. Discussion

The table below summarizes how the platform has changed relative to the 2009 framework.

Capability	2009 framework	Current platform
Client	Win32 multislice pager	MonoGame/XNA real-time client
Coordinate handling	Implicit	Explicit mosaic and volume spaces with composed STOS transforms
Annotation storage	Scalar coordinates	Indexed SQL Server spatial geometry (2015)
Annotation loading	Per section	Per spatial region (viewport)
Programmatic access	None described	OData read API; gRPC in progress
Graph export	None described	DOT, TLP, GraphML, JSON via export service
Authentication	None described	Identity service with volume-scoped claims
Assisted annotation	Manual only	gRPC segmentation (SAM2)
3D morphology	None	Bajaj surface reconstruction to COLLADA
Deployment	Lab servers	Docker, mounted-config convention

Limitations and ongoing work remain. The gRPC annotation service has not yet fully replaced the WCF implementation, so both are maintained. Mesh export to COLLADA is implemented in the morphology library but is not yet wired into the export service's public endpoints. These are areas of active development.

12. Availability

Viking and its servers are maintained in the project repository. Server components are distributed as Docker images; client builds target Windows. Access to specific volumes is governed by the identity service and granted per volume. [CITE: repository URL, license, data-availability statement.]

Notes for co-authors

- Bracketed [CITE: ...] markers indicate where formal references belong (Anderson 2009; SerialEM; ir-tools; the Bajaj contour-tiling method; SAM2; and the repository/data-availability statement).
- Figures referenced in the outline ([viking-methods-outline.md](#)) are not yet drawn; the architecture diagram there can seed Figure 1 and Figure 3.
- Quantitative scale figures (volume sizes, annotation counts, annotator numbers) should be filled from current project records; the original RC1 figures are in Anderson 2009.
- All technical claims here are grounded in the current source tree at the cited paths; verify against the specific release used for the manuscript.